WELCOME TO THE COURSE ON

Lect.1

MICROPROCESSORS AND MICROCONTROLLERS LAB (BECE204P)

by

Abdul Rahim V C

School of SENSE
Department of Communication Engineering
Vellore Institute of Technology, Vellore

Course Objectives

1. To familiarize the students with assembly language programming using microprocessor and microcontroller.
2. To familiarize the students with Embedded C language programming using microcontroller.
3. To interface peripherals and I/O devices with the microcontroller and microprocessor.

Course Outcome

1. Showcase the skill, knowledge and ability of programming microcontroller and microprocessor using its instruction set.

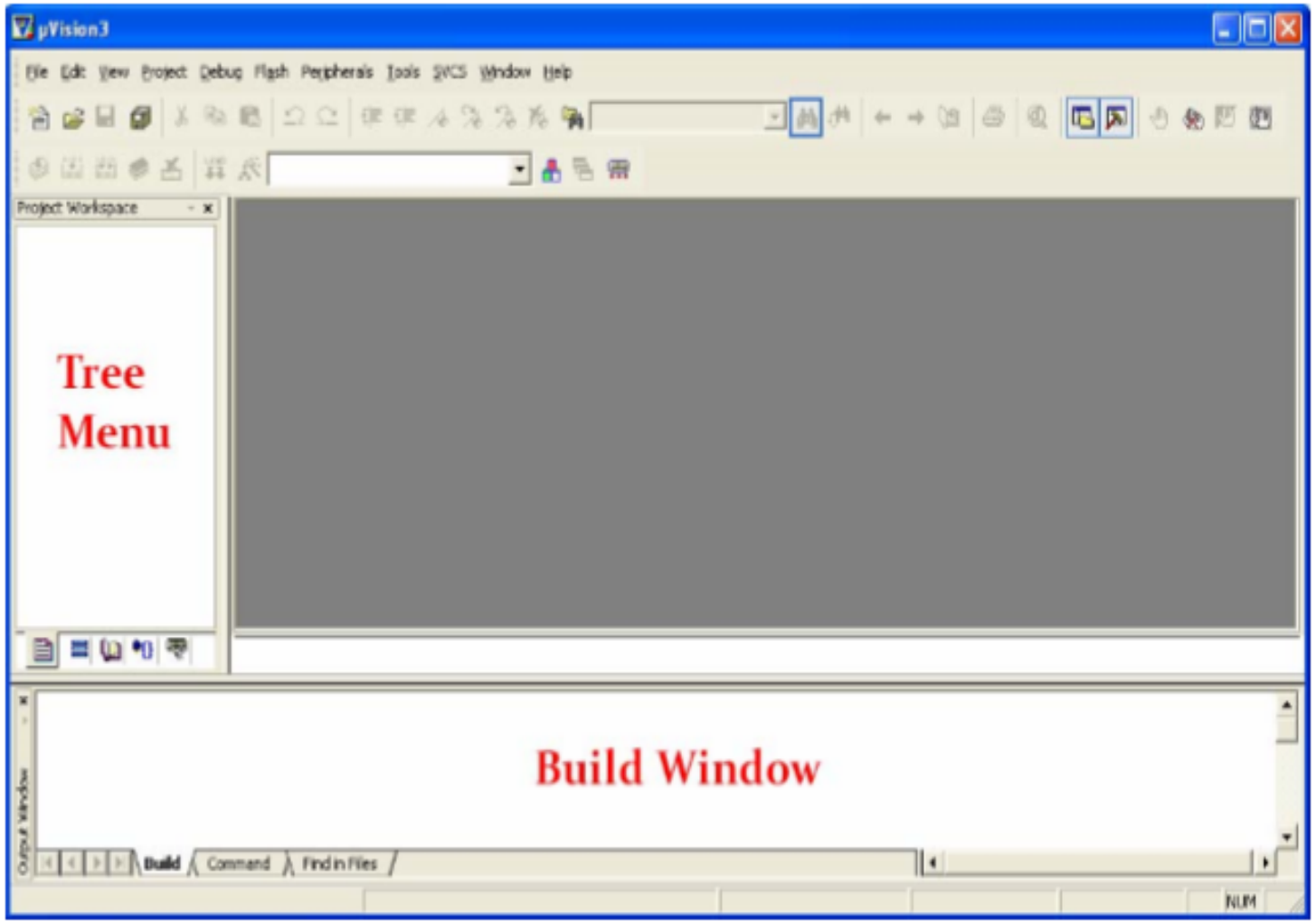
2. Expertise with microcontroller and interfaces including general purpose input/ output, timers, serial communication, LCD, keypad and ADC.

Assessment Rubrics

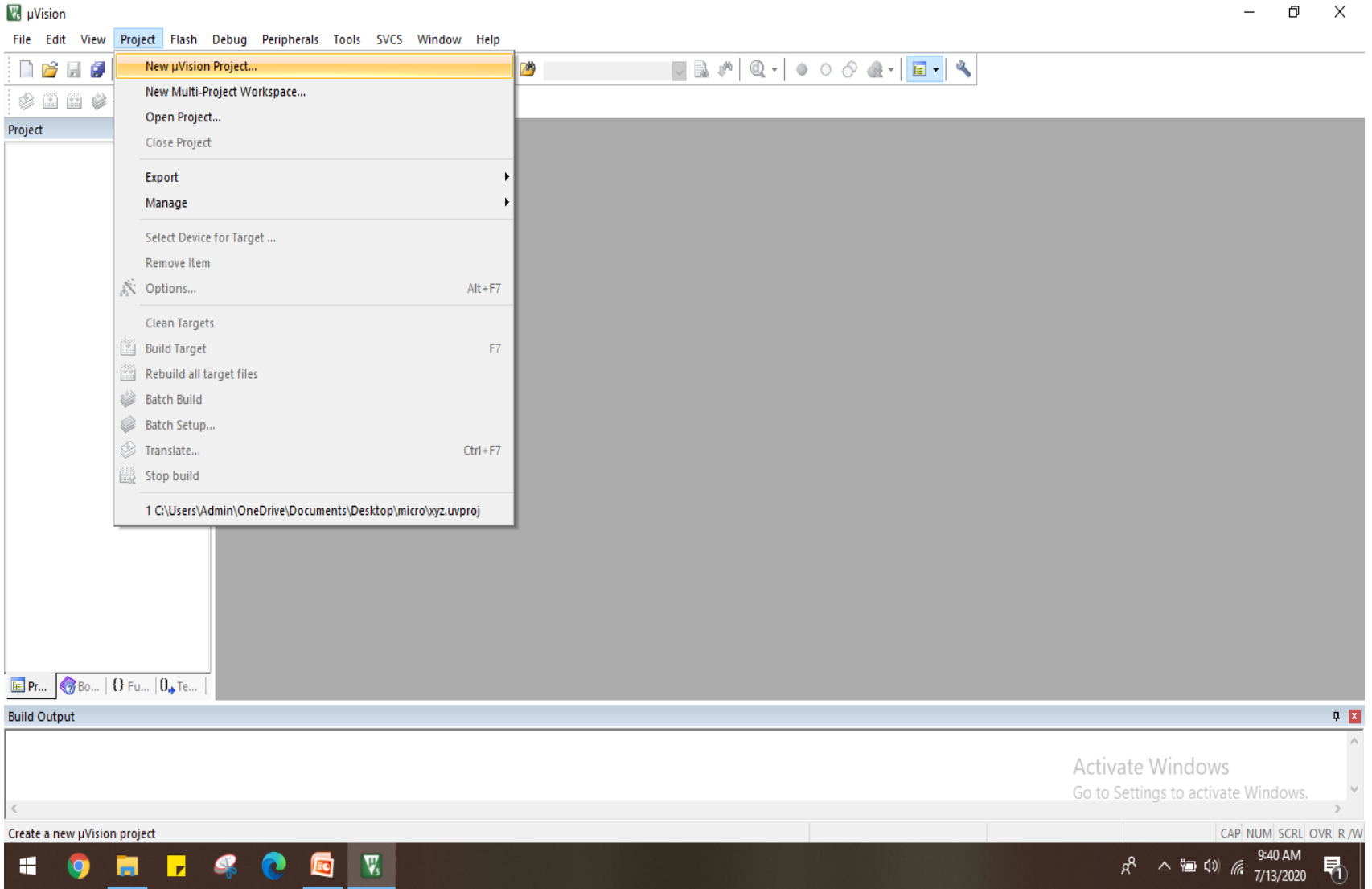
Assessment Number	Title	Maximum mark	Weightage	Due date
1	General assembly programming	12	12	09-02-2026
2	I/O port programming	12	12	16-02-2026
3	Timers and Counters	12	12	09-03-2026
4	Serial Communication	12	12	30-03-2026
5	Interrupts	12	12	10-04-2026
	FAT - Final Assessment Test	50	40	-

Instructions to use KEIL SIMULATOR

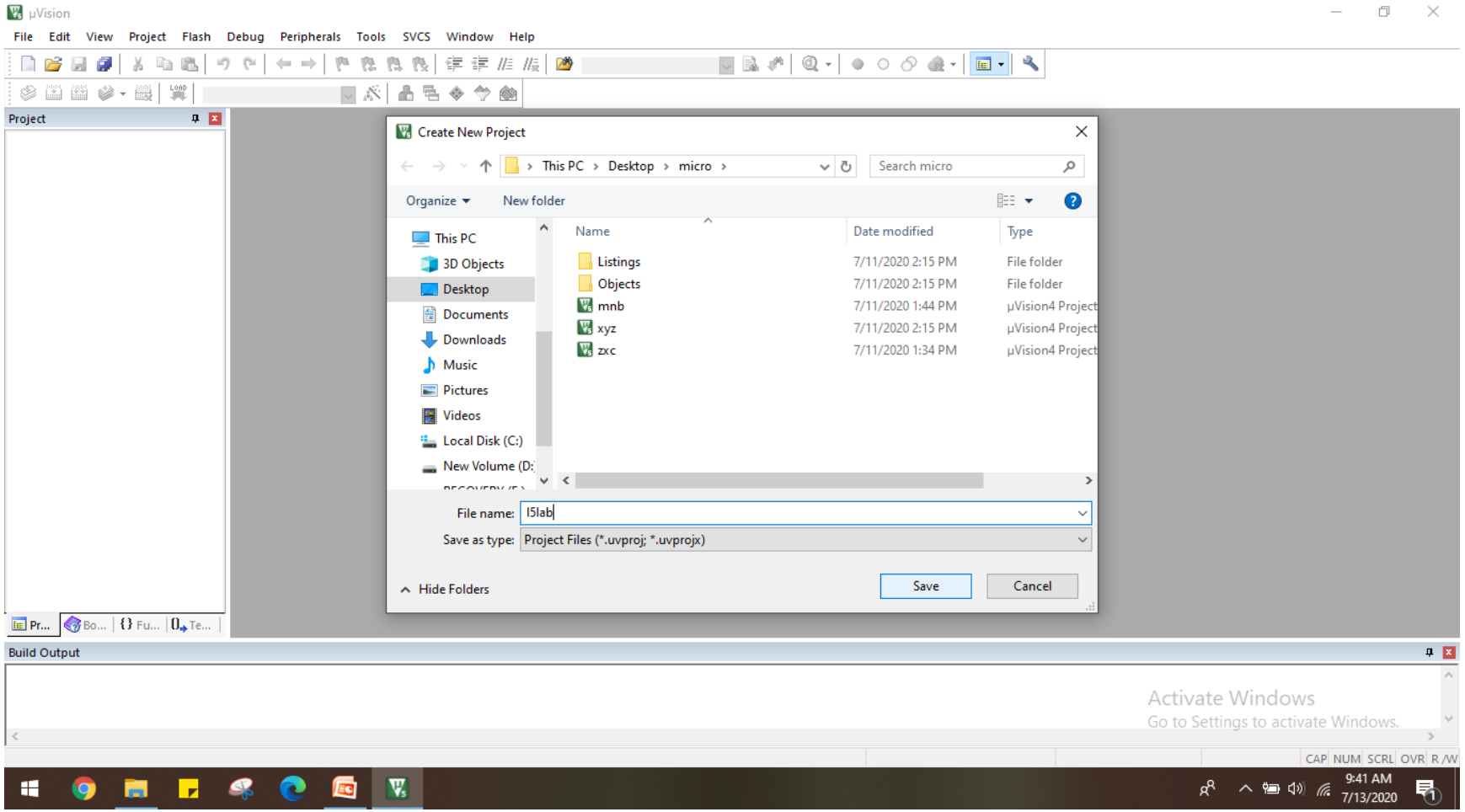
STEP 1:Open Keil uVision5
(Start >Programs>Keil uVision 5)



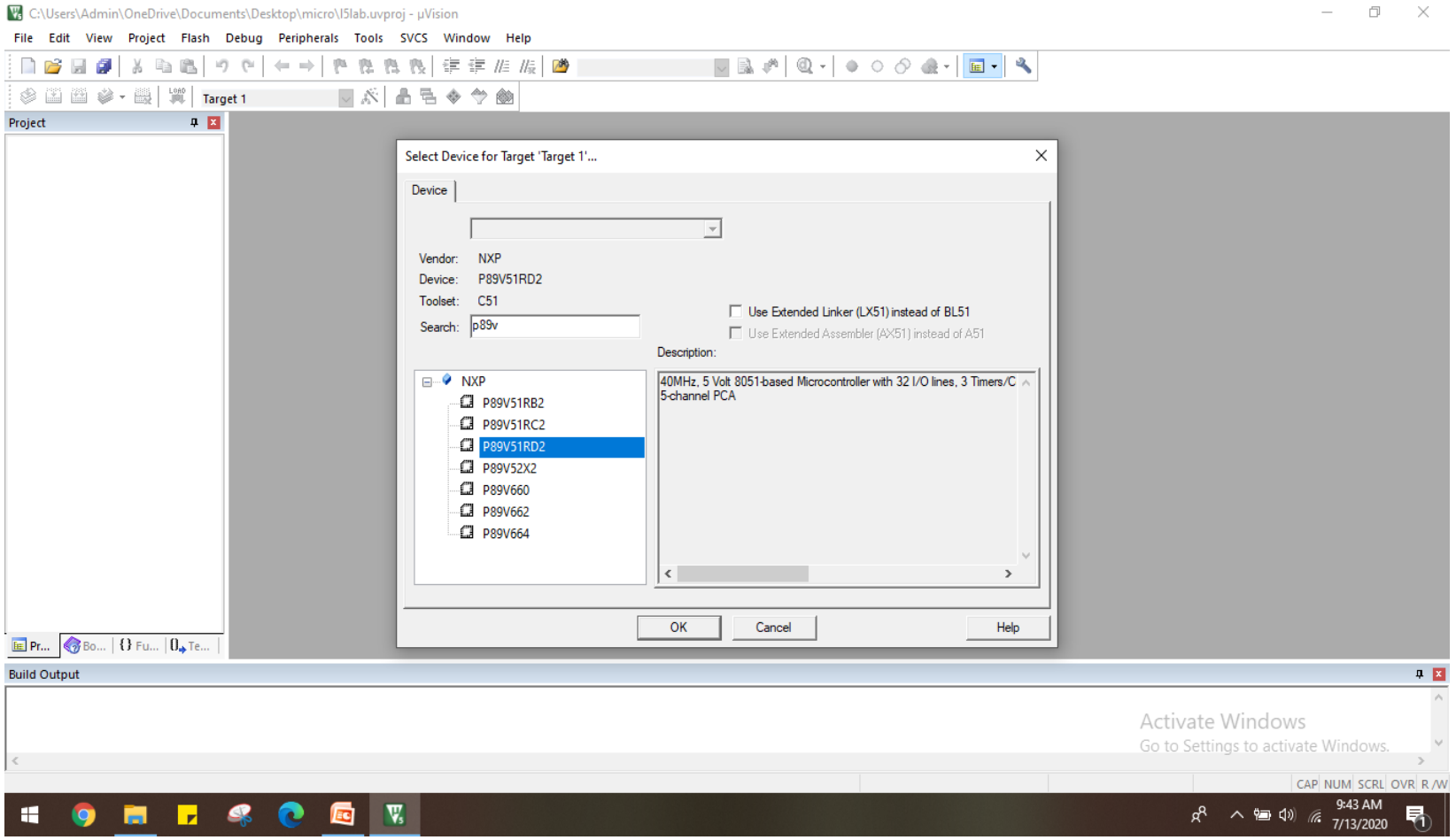
STEP2:Start a new project (File>New>µvision project



STEP3:Give any name without extension, and Save in your folder

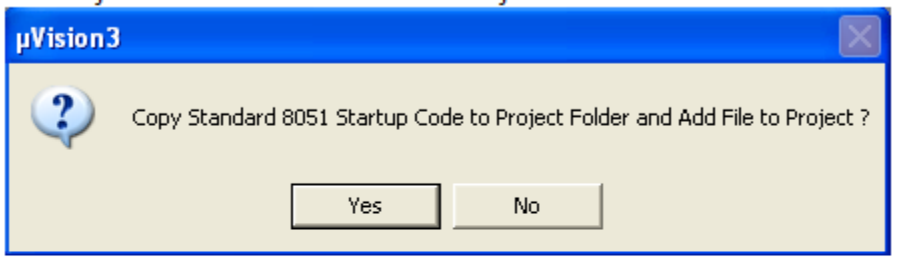


STEP 4:Select Nxp>P89V51RD2

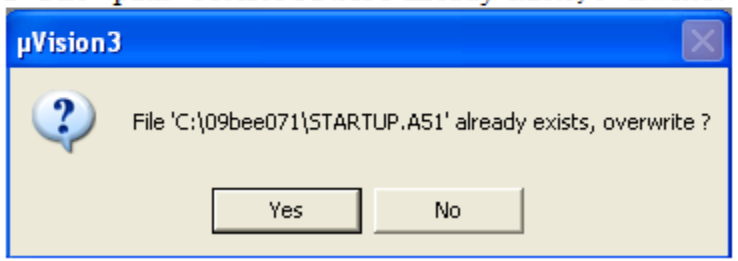


STEP 5(select→yes)

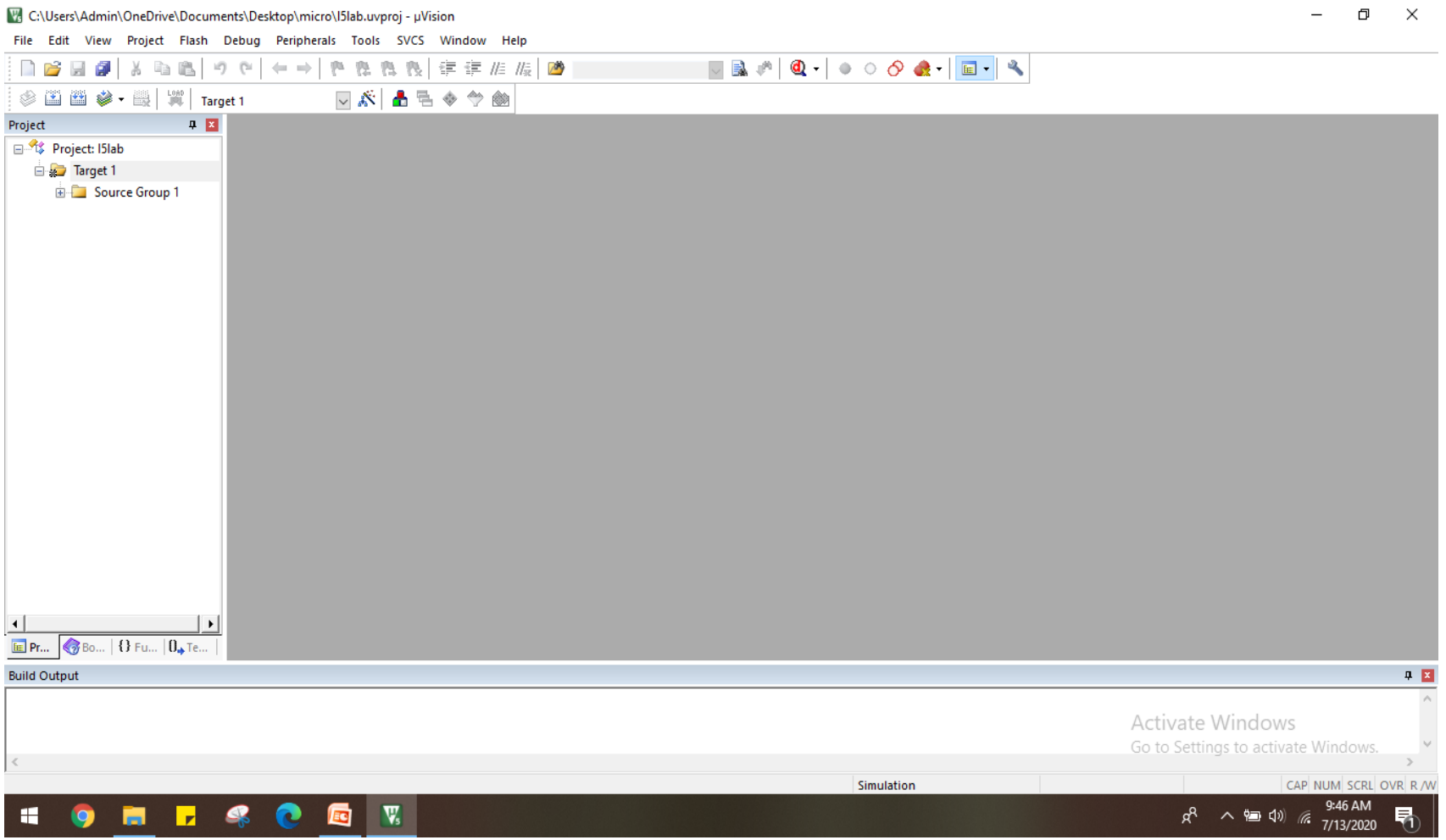
- Click Yes when asked whether to "Copy Standard 8051 Startup Code to Project Folder and Add File to Project."



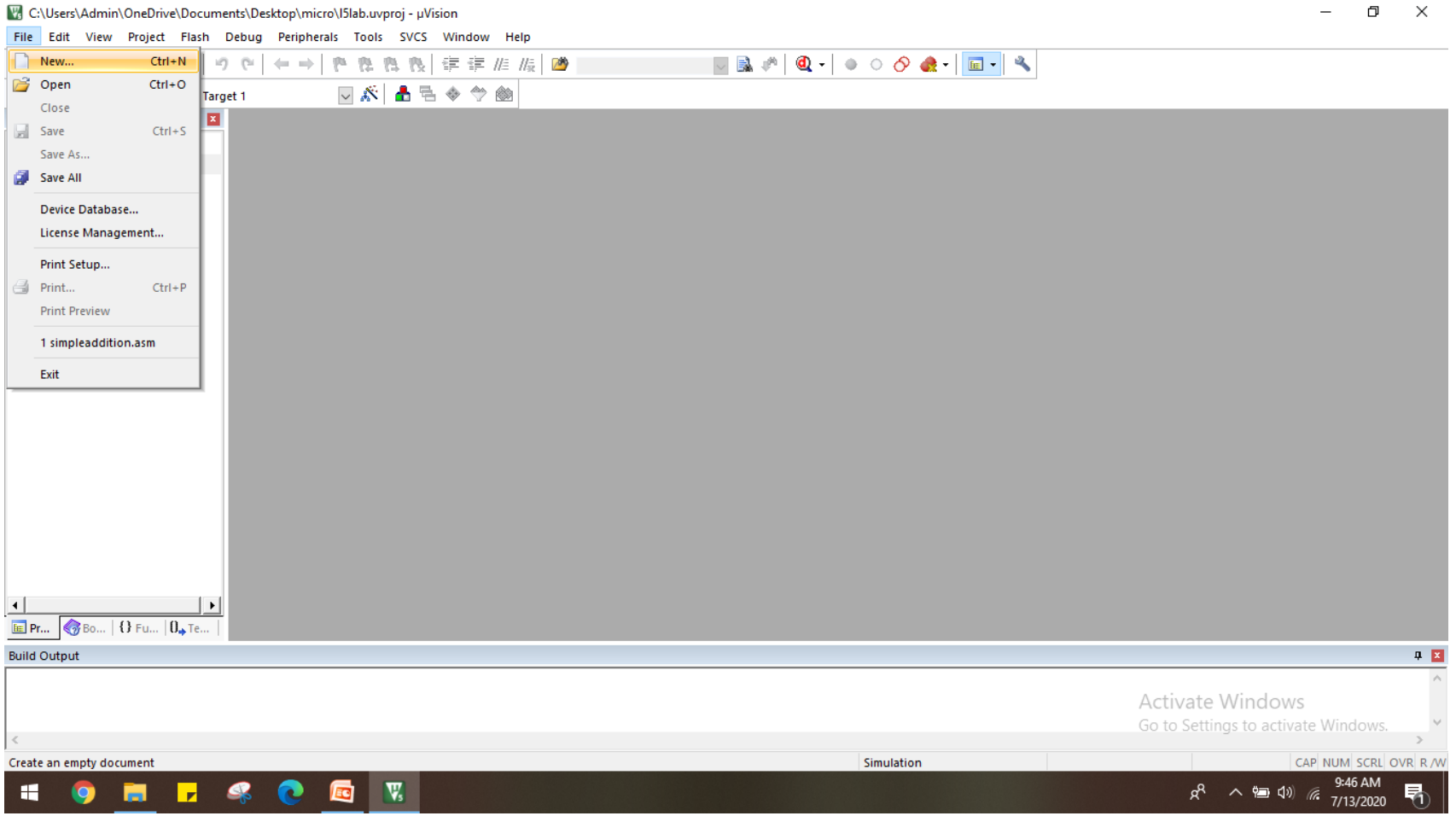
- Click Yes if asked "File <path>STARTUP.A51 already exists, overwrite?



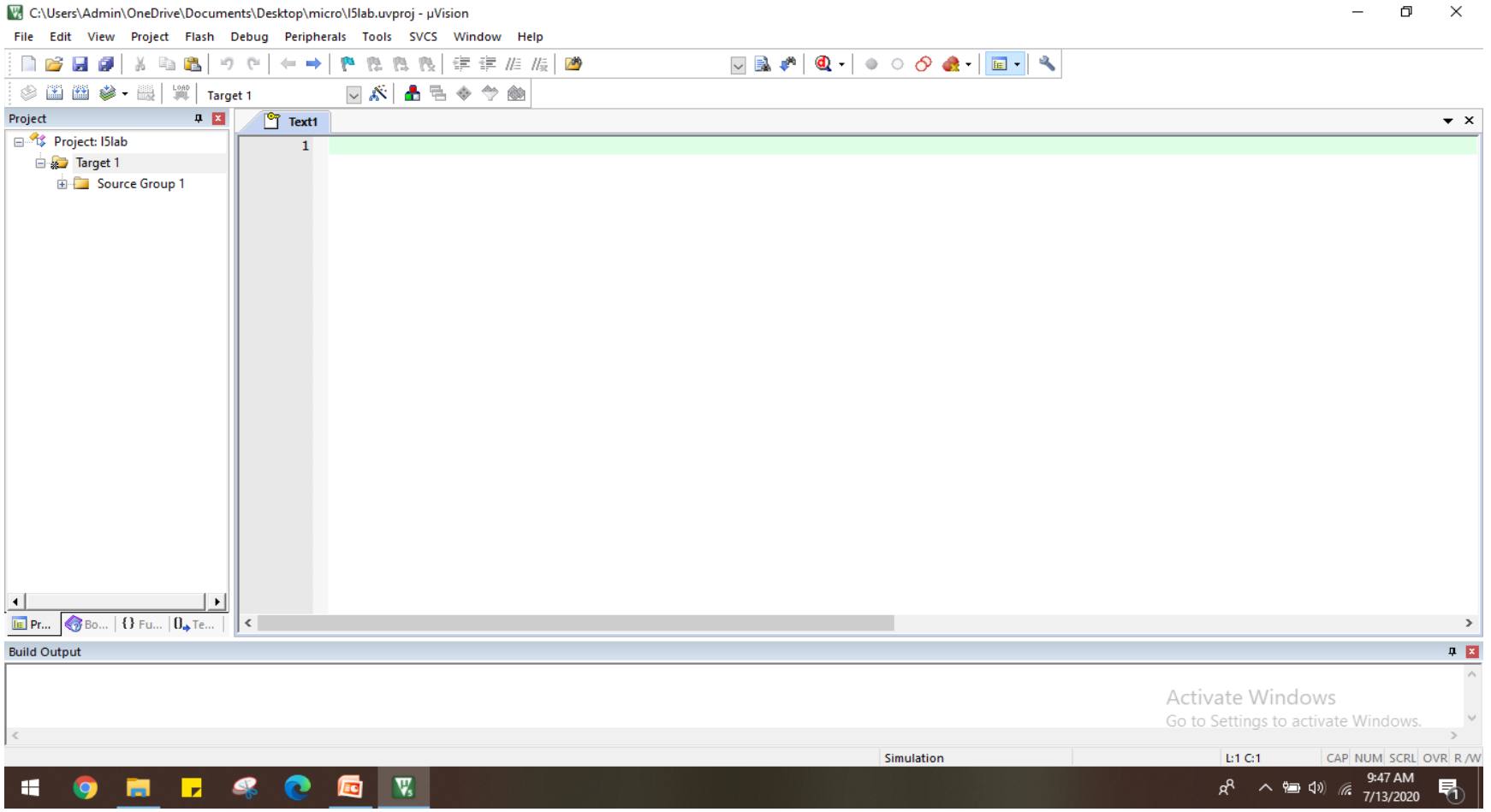
Blank space will appear



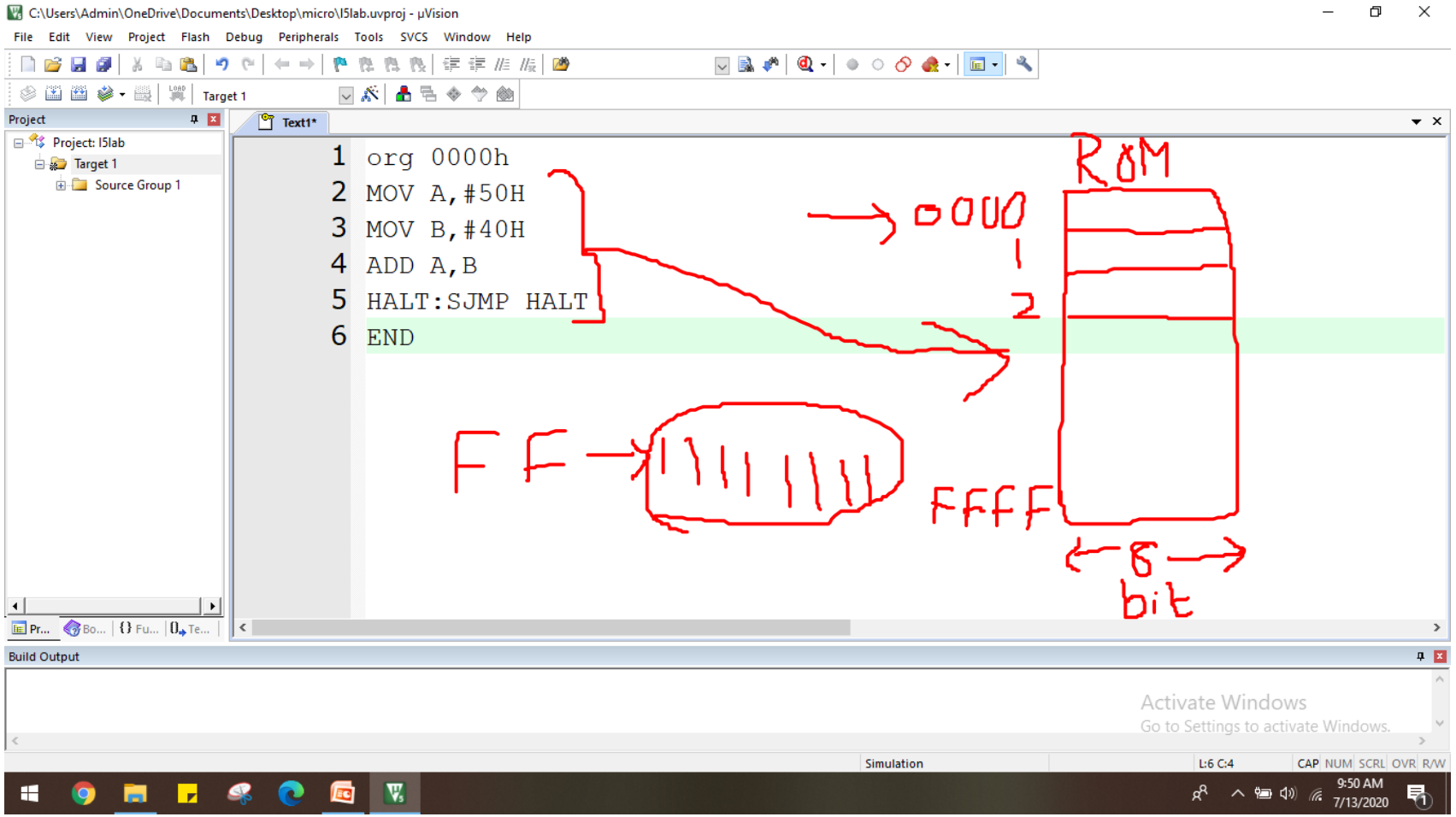
STEP 6:To create source file
Open a new file (File→New File)



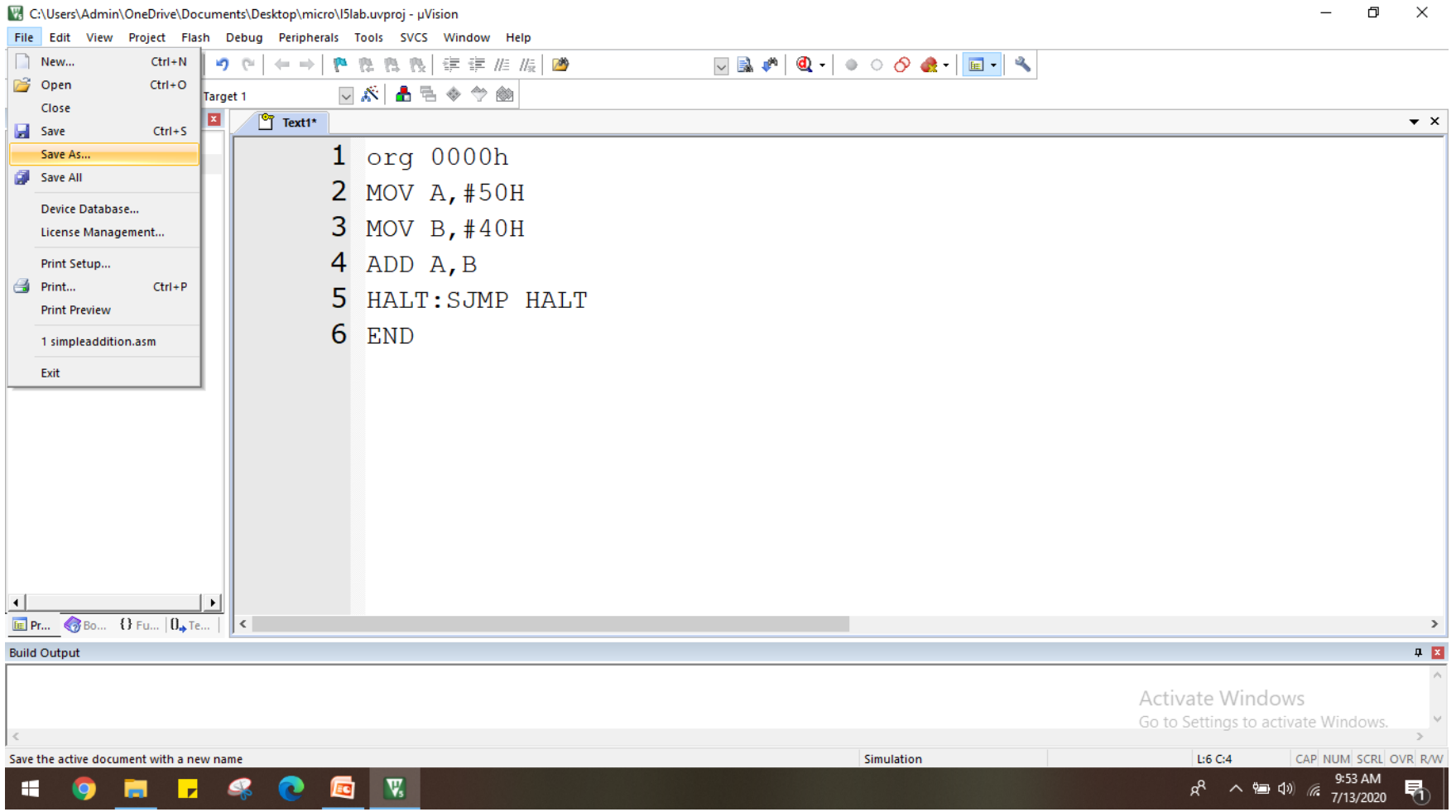
STEP 6:A new window will open with blank space to type the code



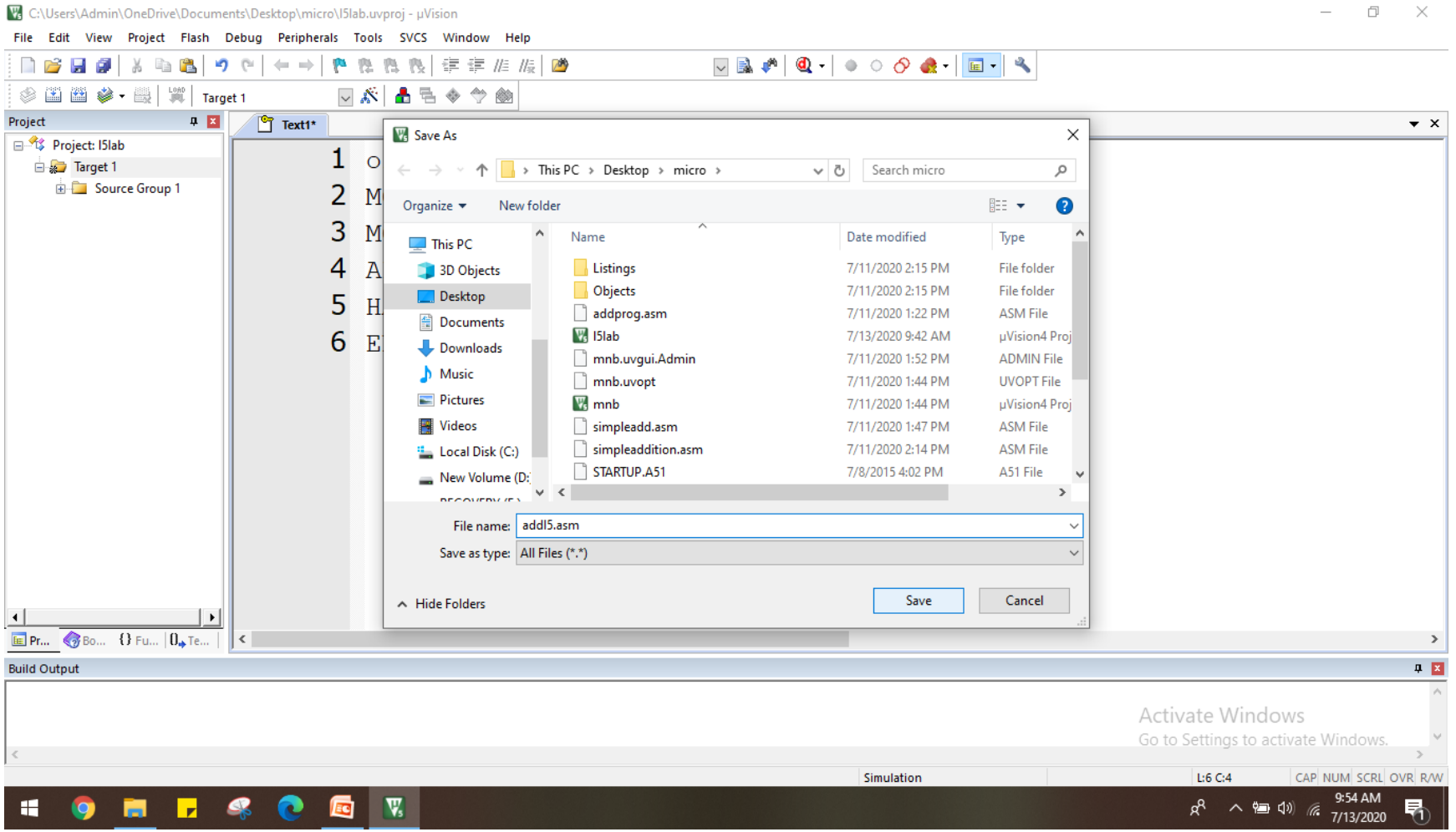
STEP 7: TYPE THE CODE IN BLANK SPACE



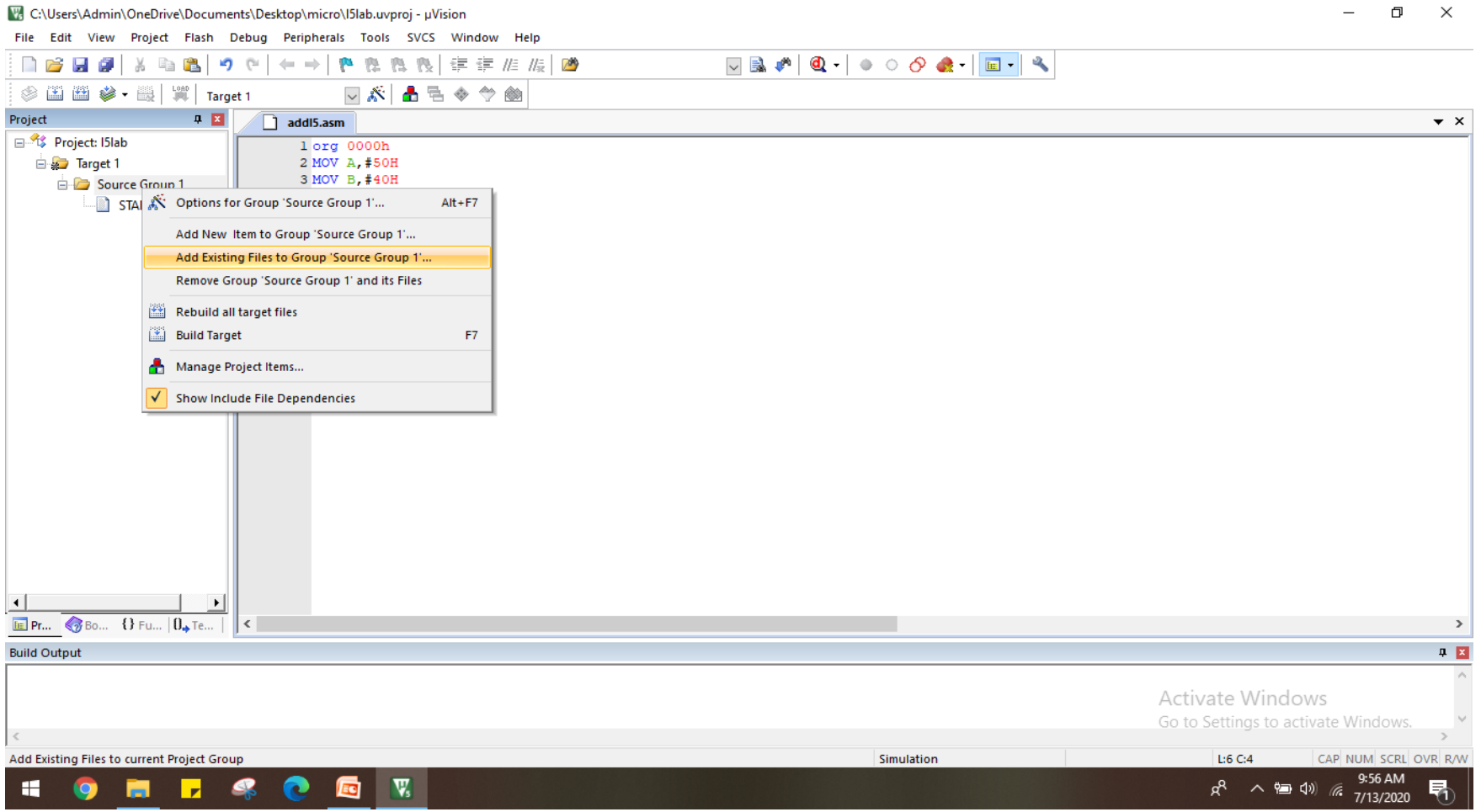
STEP 8:Click file menu and save



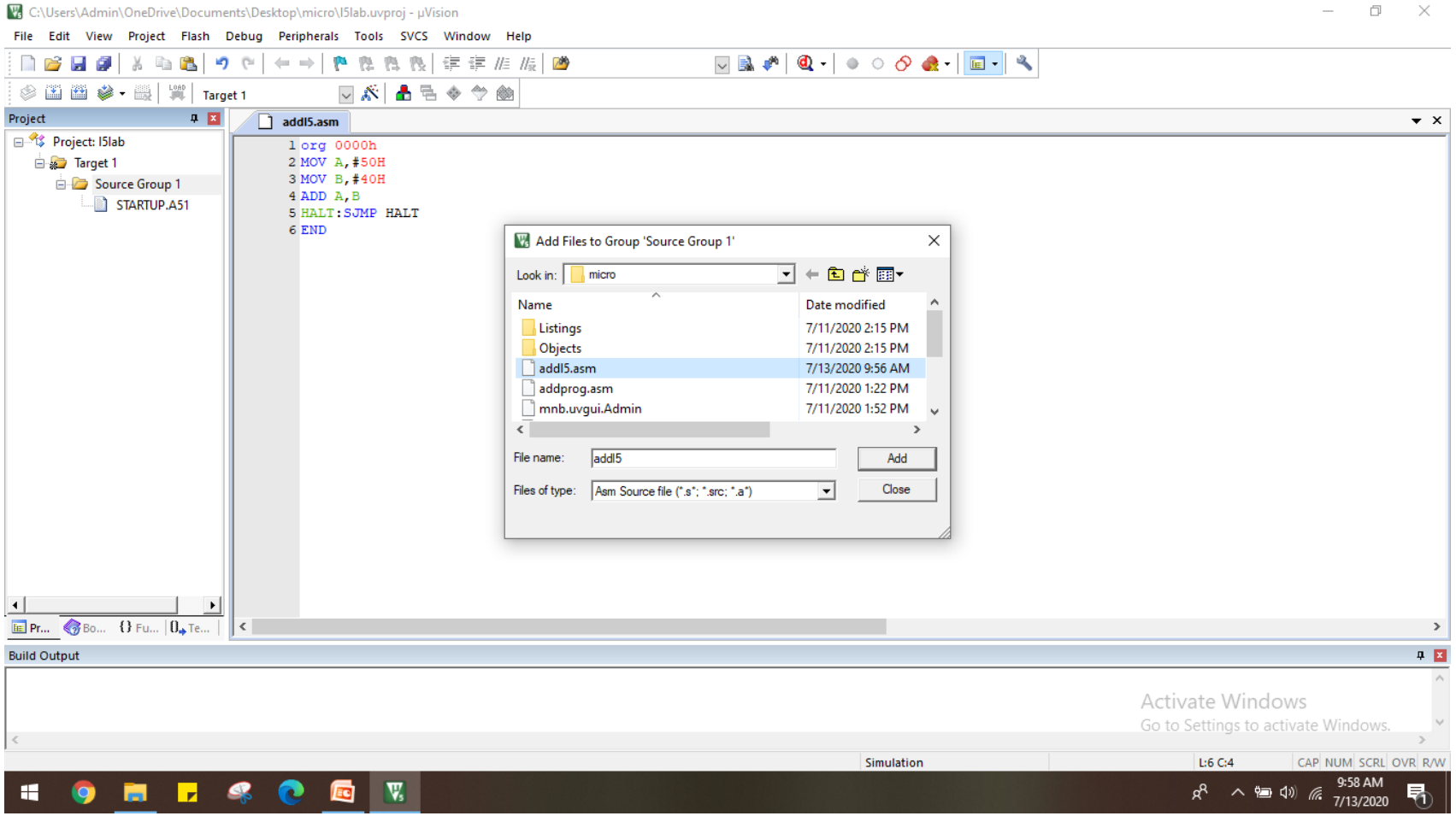
STEP 9:Save the file name with extension as →.asm



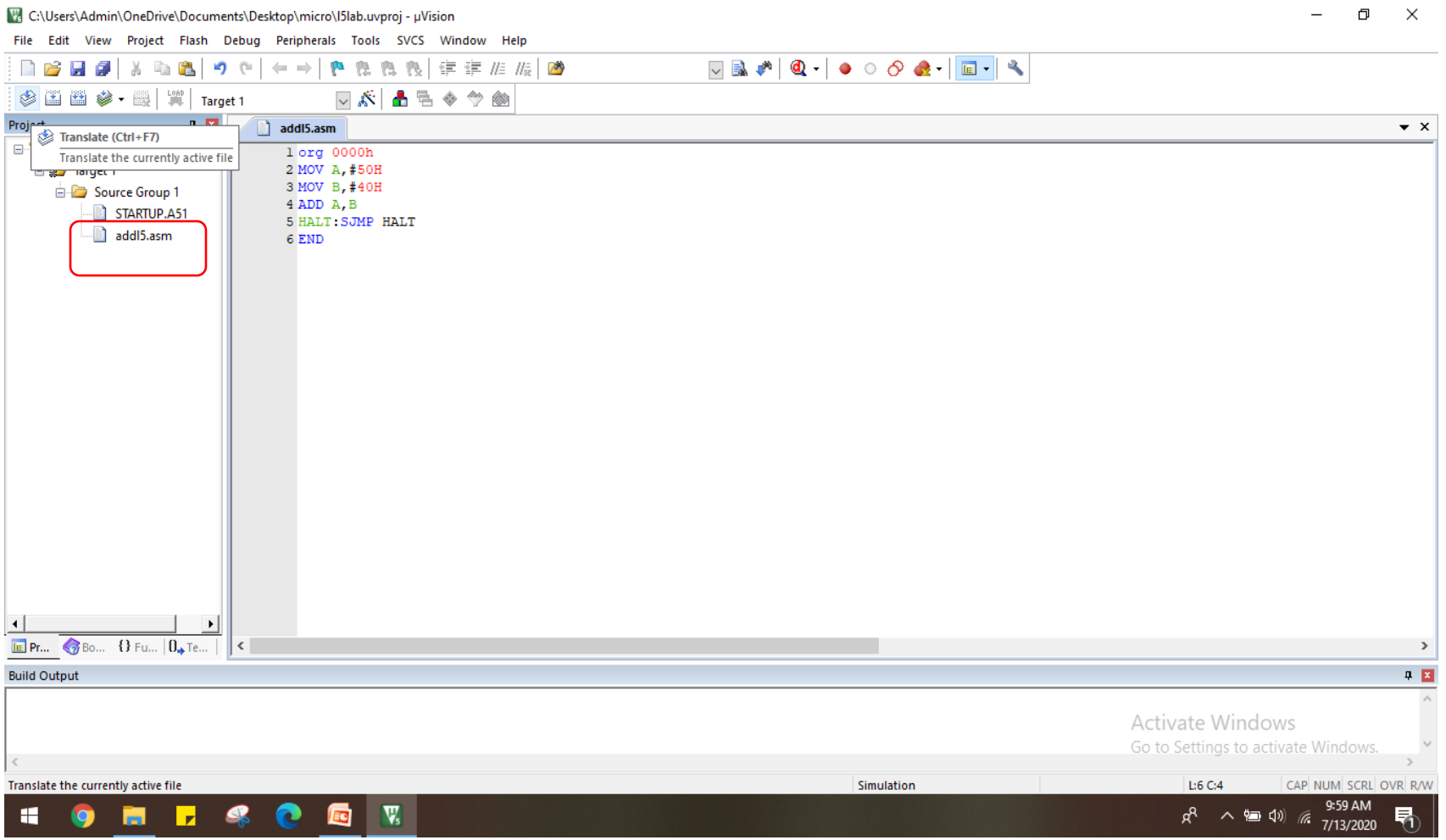
Step 10: Right click the source group and select→Add existing file to the source group



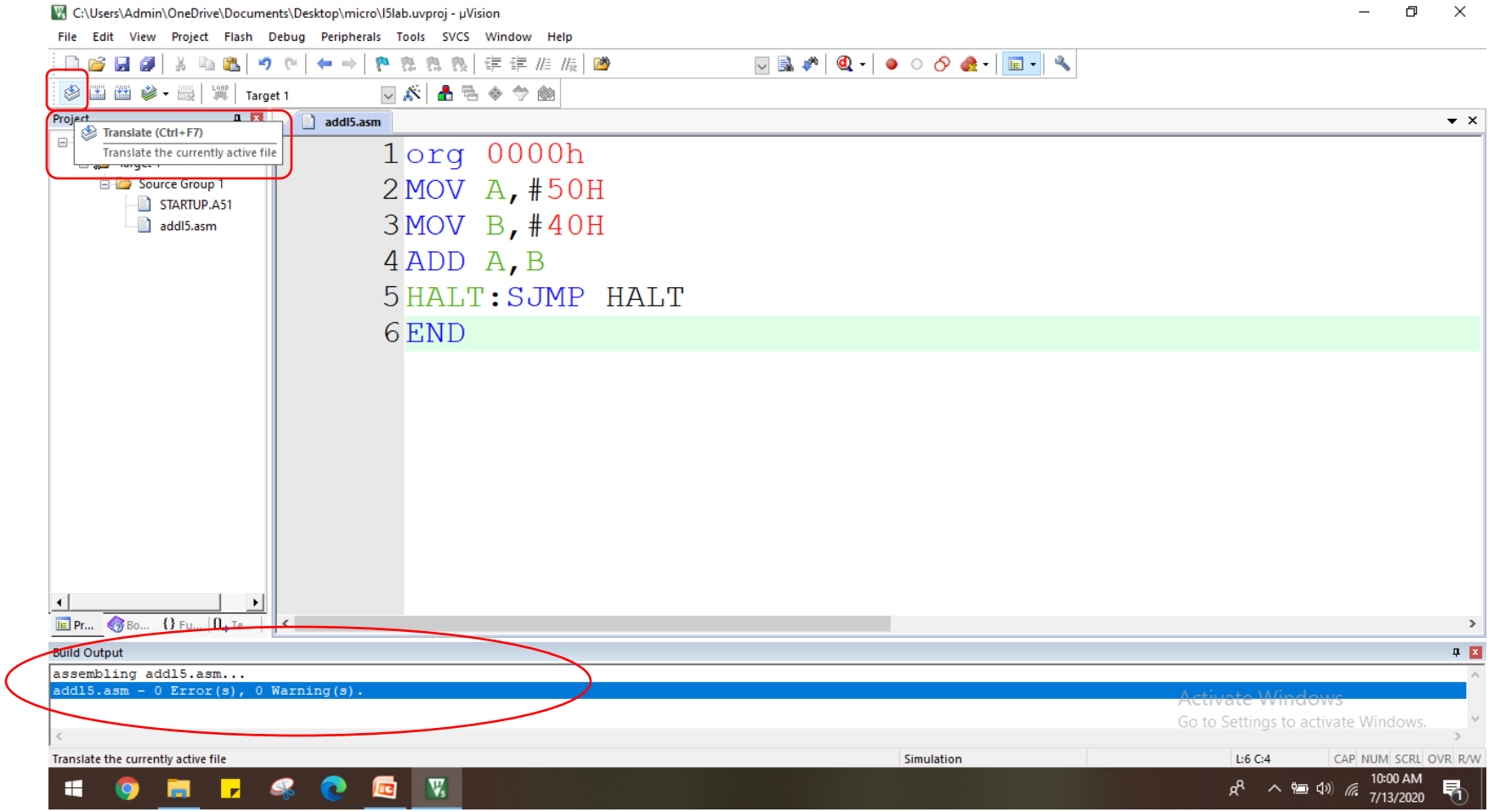
Step 11: select the saved file from the folder and click→ADD and close



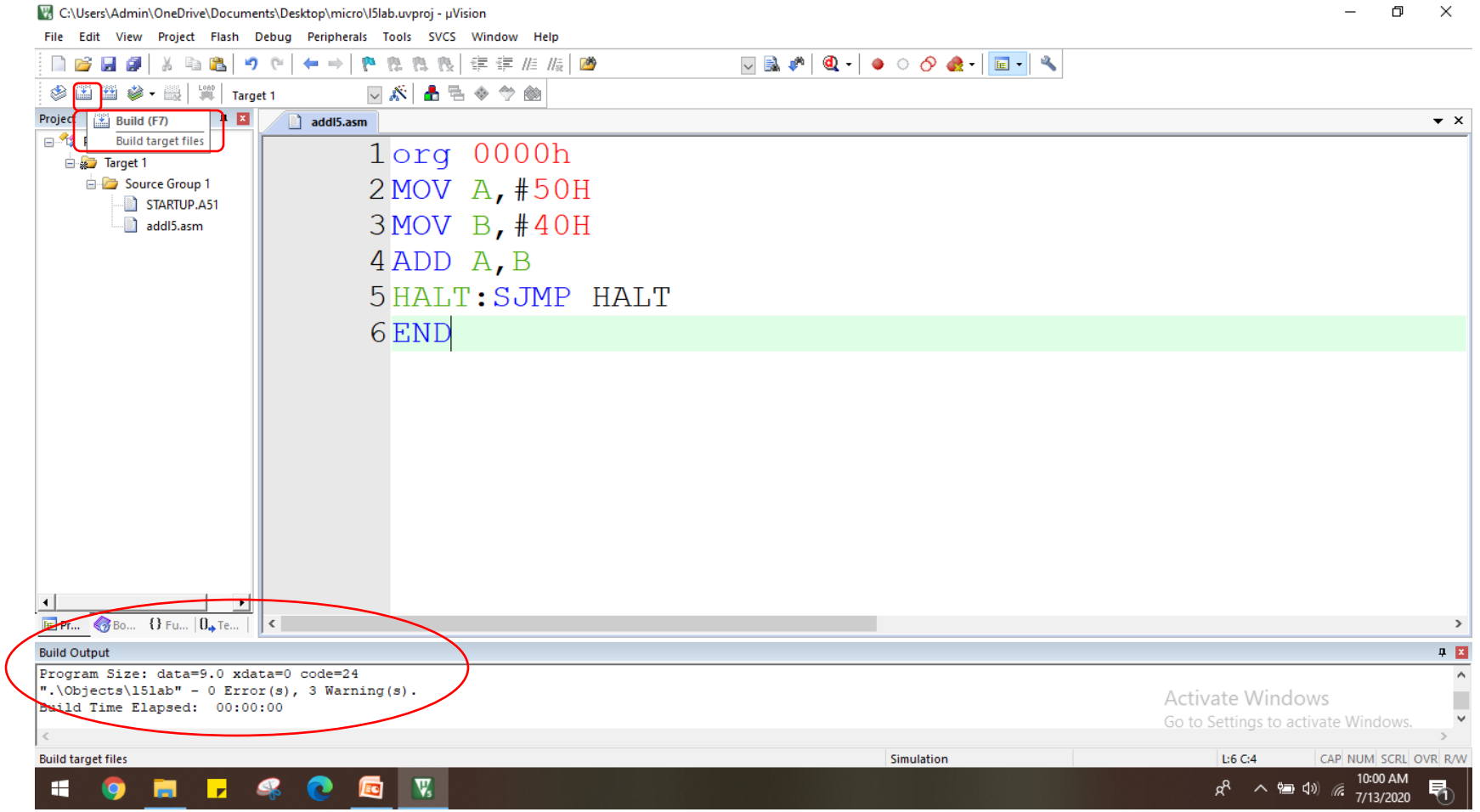
Step12:Expand the source group in project work space and ensure your file is under source group



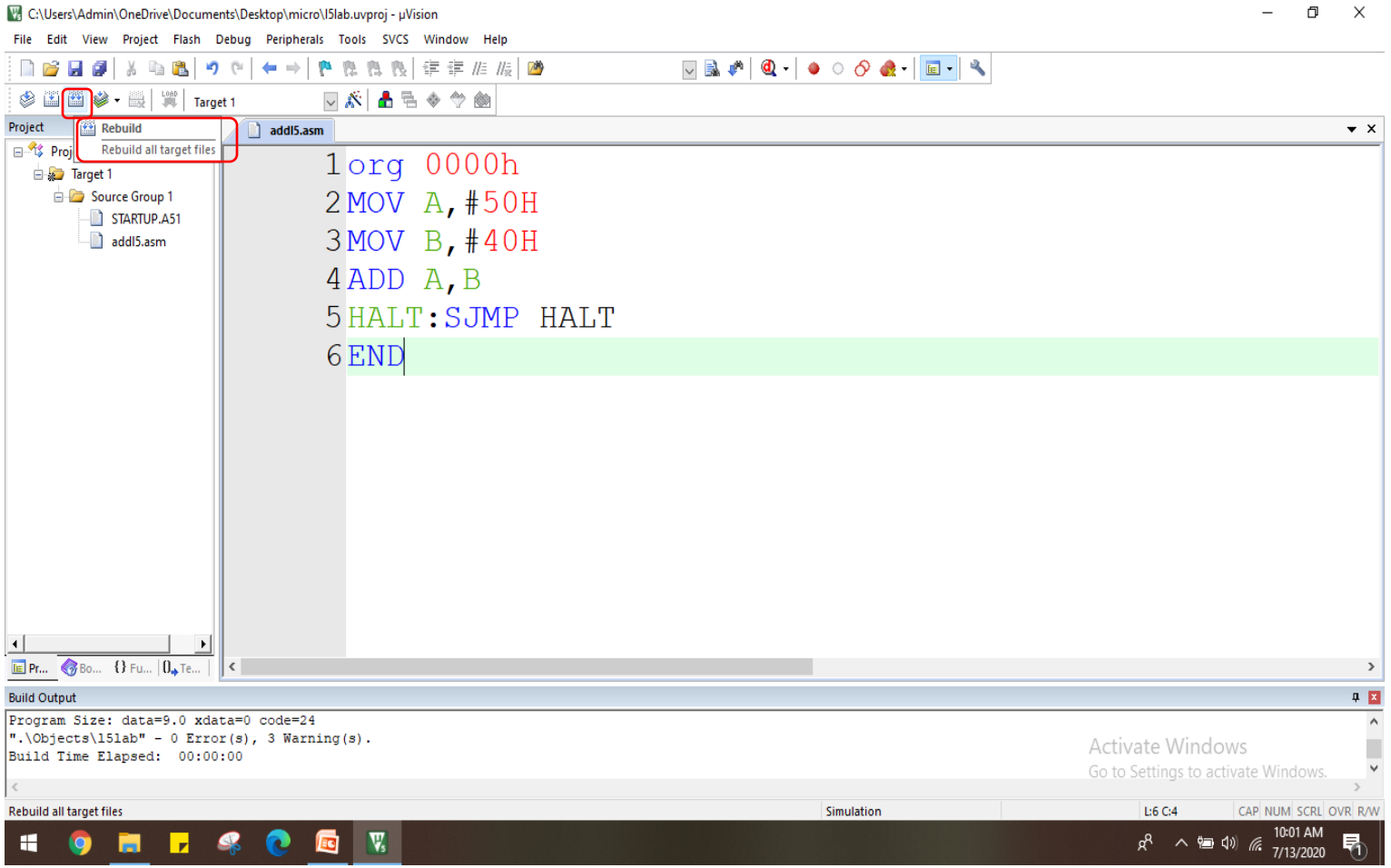
Step13: Select→”Translate” on the left corner and check for syntax Errors



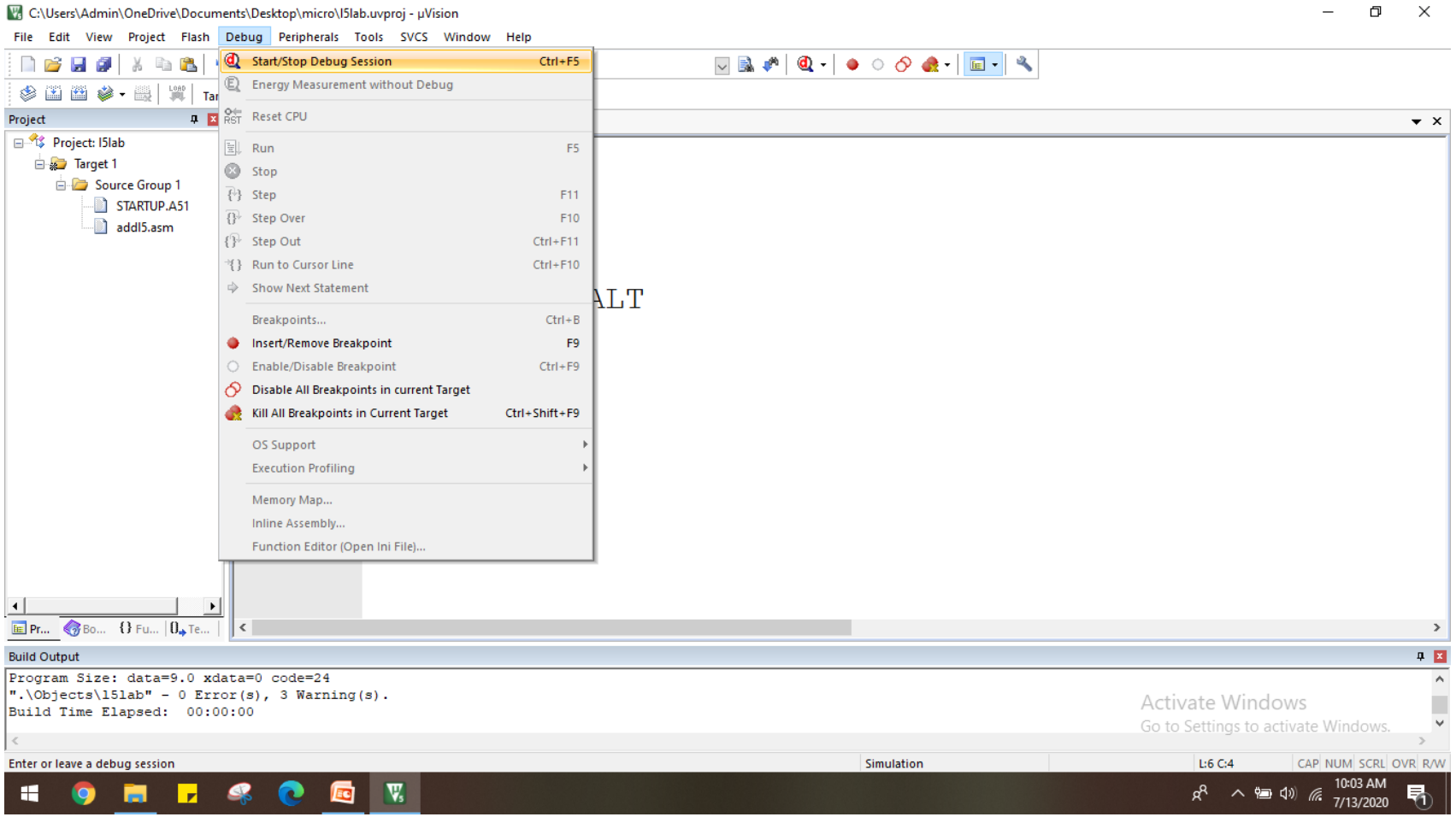
Step14: Select→”Build” on the left corner



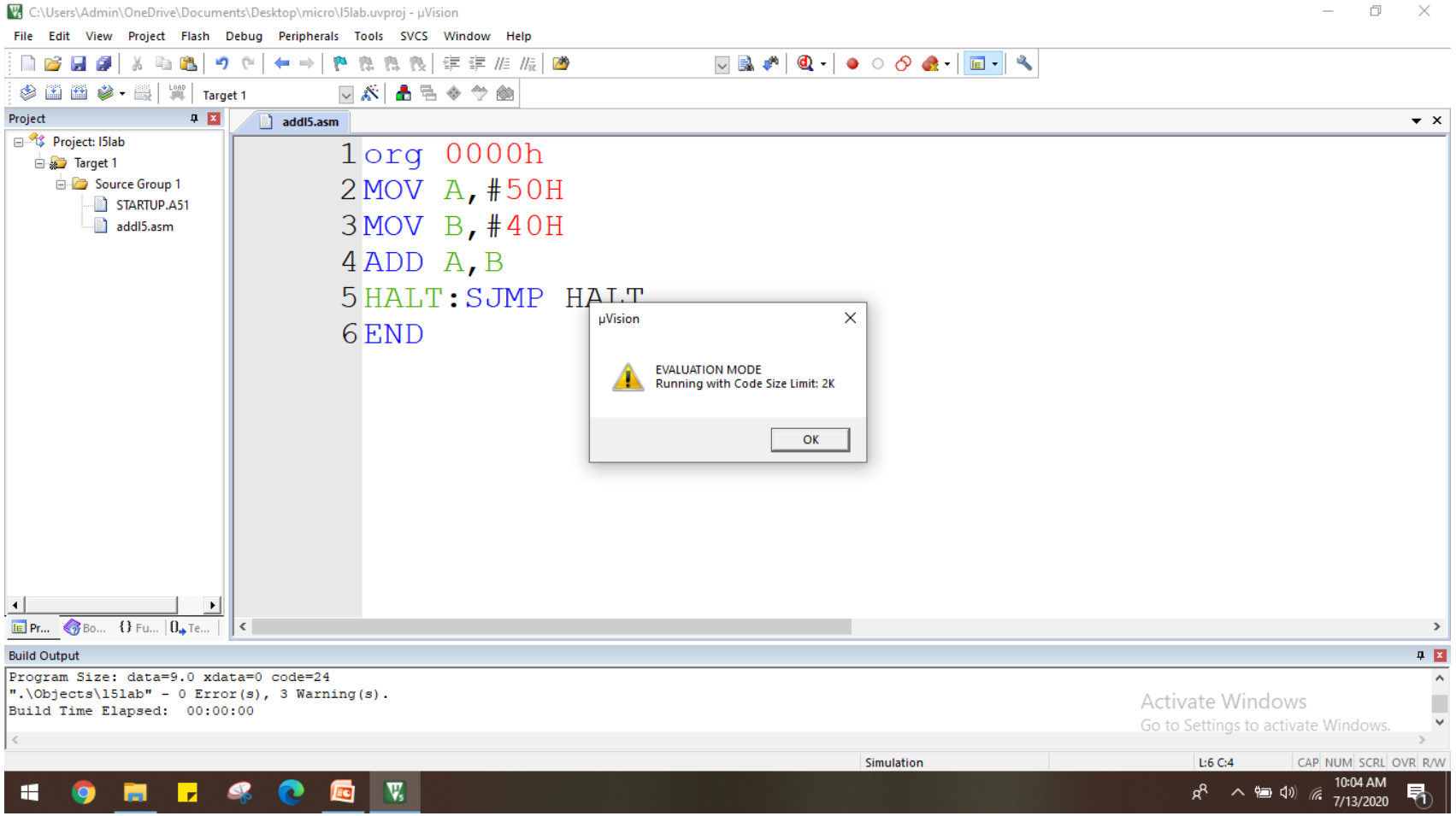
Step13: Select→”Rebulid” on the left corner



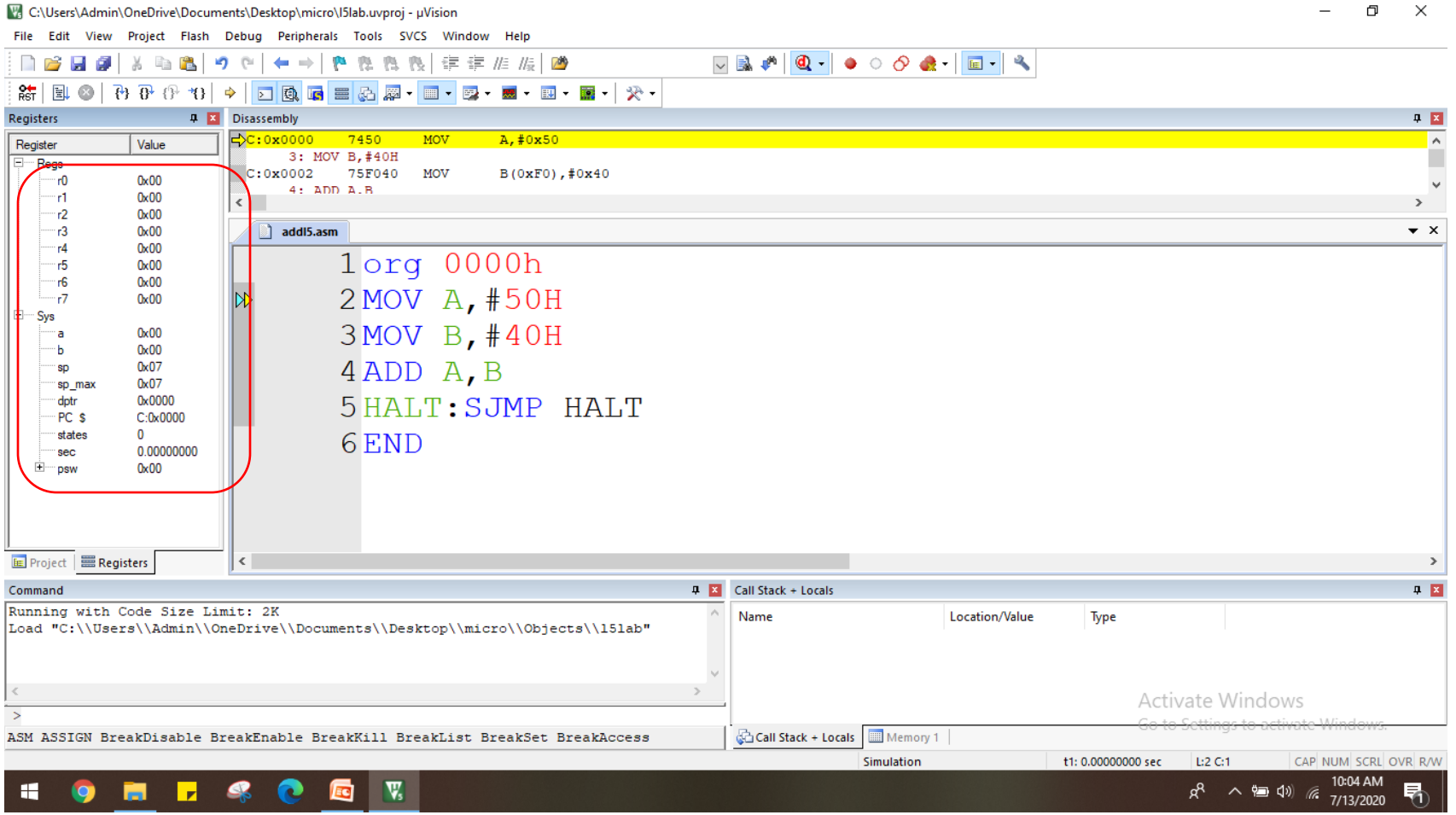
Step14:Debug→Start Debug



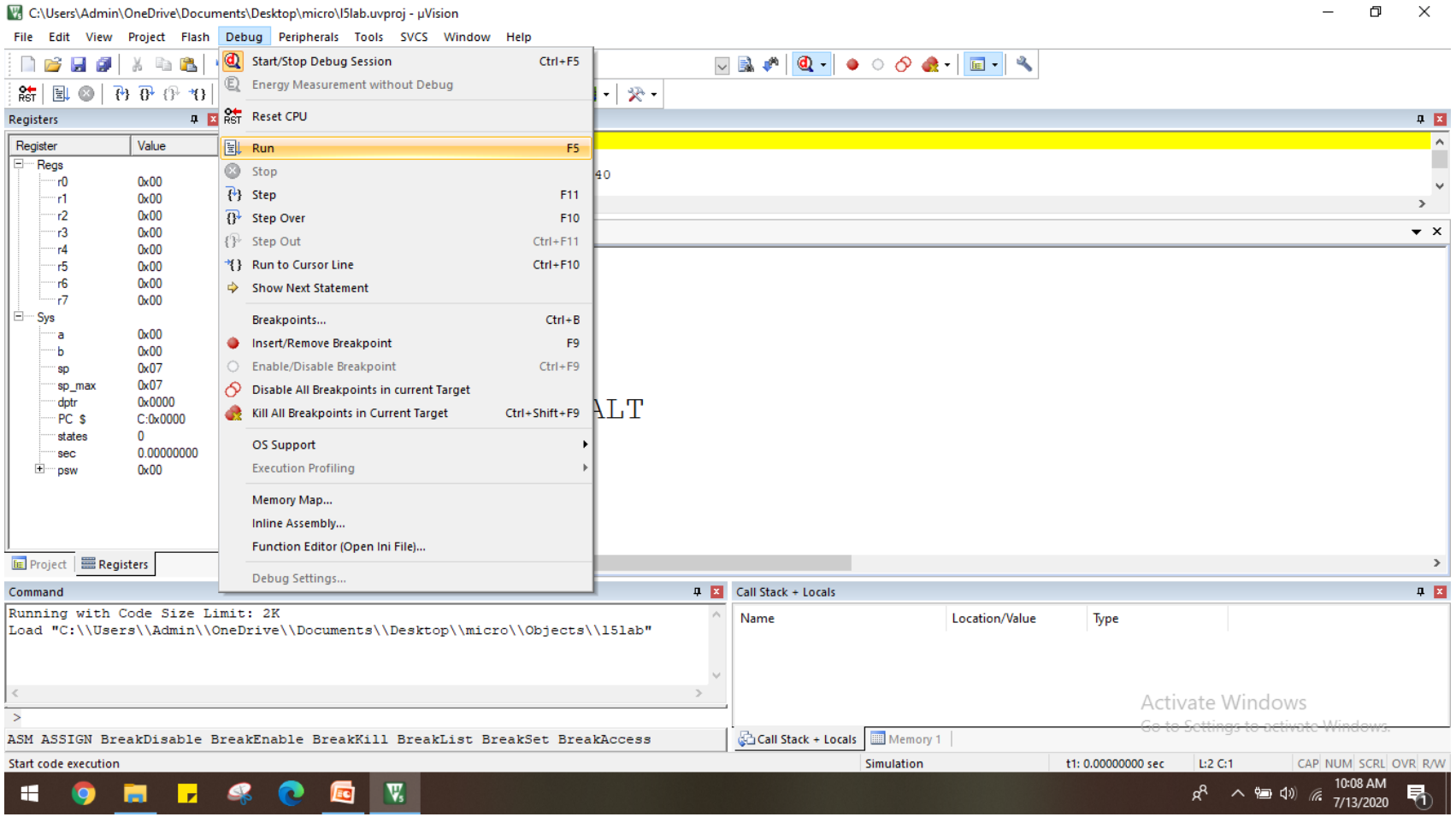
Step15: click →"ok"



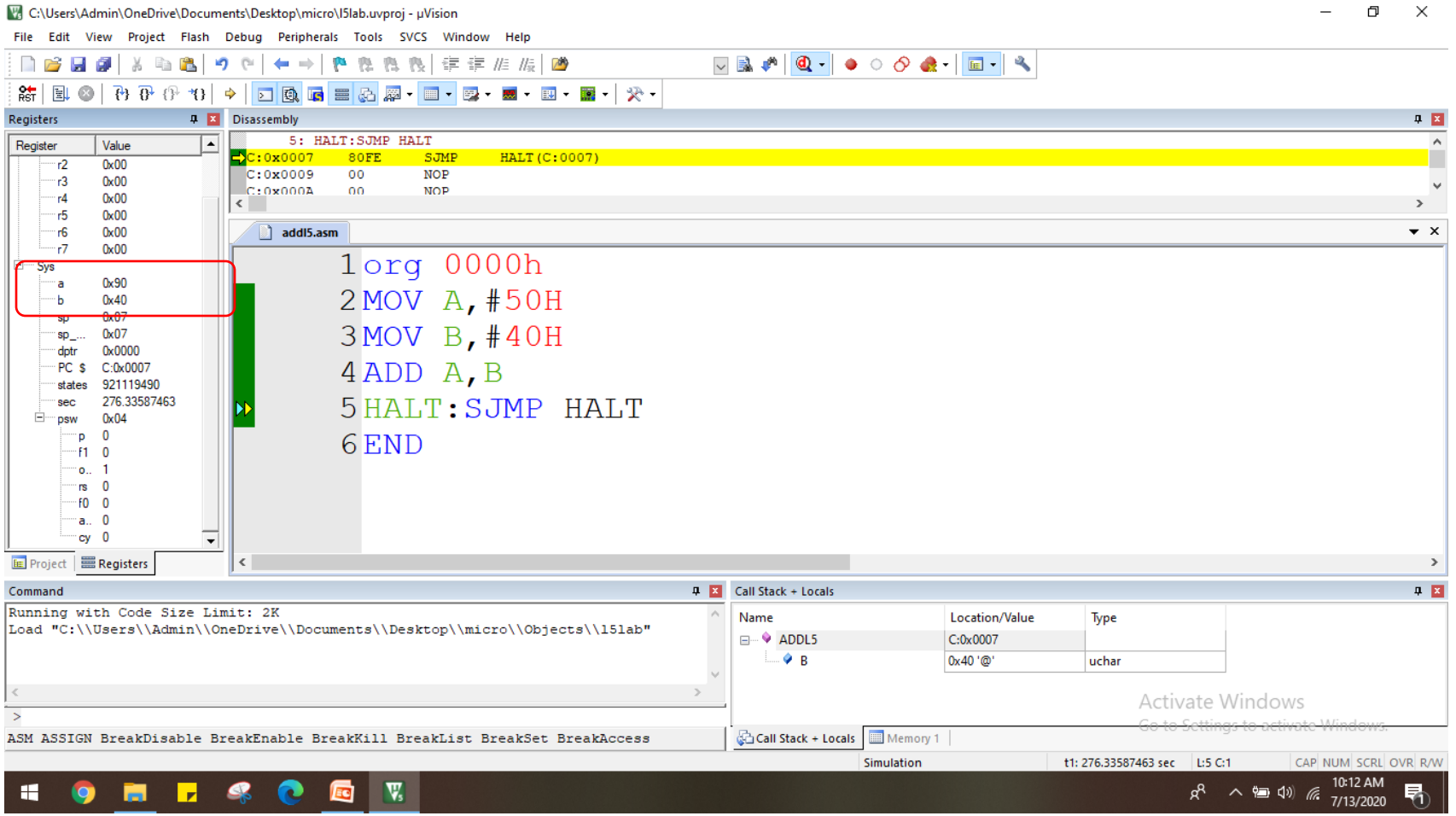
Registers will appear on the left



Step16:click Debug→Run



Check the results



MICROPROCESSOR & MICROCONTROLLER LAB RULES

1. Maintain a Separate Lab Notebook

- You must have a **separate notebook** exclusively for this lab.
- Write **each experiment clearly** (Aim, Program, Output screenshot space).
- After completing an experiment, you must get it **signed by the TRA** with **date**.

MICROPROCESSOR & MICROCONTROLLER LAB RULES

2. Mandatory Signatures

- You cannot skip signatures.
- We will randomly collect lab notebooks to verify the signatures.
- Missing signatures = experiment considered **incomplete**.

MICROPROCESSOR & MICROCONTROLLER LAB RULES

3. Lab Template Submission (VTOP)

- I will share a **template** for every experiment.
- After completing the experiment:
  - Paste **Register window before execution**
  - Paste **Register window after execution**
  - Paste **photo of TRA signature + written program** from notebook
- Convert to **PDF** and upload to **VTOP** before the deadline

MICROPROCESSOR & MICROCONTROLLER LAB RULES

4. Strict Academic Integrity

- No copying programs from others.
- No sharing screenshots, copying signatures, or exchanging notebooks.
- If cheating is found, experiment marks = **0**.
- Be honest

MICROPROCESSOR & MICROCONTROLLER LAB RULES

5. Lab Software Availability

- Keil µVision software will be provided.
- You may **install it on your laptop** and practice at home/ hostel.
- Bring your laptop (OPTIONAL) if you want to try during lab time.

MICROPROCESSOR & MICROCONTROLLER LAB RULES

6. Bring Required Items

- Lab notebook
- Pen/pencil
- Laptop (optional)
- Good attitude & willingness to learn 🙂

MICROPROCESSOR & MICROCONTROLLER LAB RULES

7. Maintain Lab Discipline

- Don’t misuse lab computers.
- Don’t change system settings or delete files.
- Keep your workstation clean.

MICROPROCESSOR & MICROCONTROLLER LAB RULES

8. Save Your Work

- Save your Keil projects inside a folder with your **register number**.
- Don’t leave files on desktop or shared common folders.

Write an ALP to perform addition of two 8-bit numbers

- ORG 0000H ; ensures that your first instruction is placed where the CPU begins execution after reset.
- MOV A, #30H ; load 30H (hexadecimal) value to accumulator
- MOV B,#25H ; load 25H (hexadecimal) value to register B
- ADD A,B ; A = A + B
- H1: SJMP H1 ; an infinite loop stops the microcontroller from running into garbage.
- END ; END stops the assembler from reading more code.

Write an ALP to perform addition of two 8-bit numbers

Label	Mnemonics	Operands	Comments	Final PSW & Register Contents
—	ORG	0000H	Program begins at reset address	—
—	MOV	A,#30H	Load 30H into accumulator	A = 30H
—	MOV	B,#25H	Load 25H into register B	B = 25H
—	ADD	A,B	Add A+B, and result stored in A	A = 55H, CY=0
H1	SJMP	H1	Infinite loop to stop program	PSW = 00H, A=55H, B=25H
—	END	—	End of source file	—

Write an ALP to perform subtraction of two 8-bit numbers

- ORG 0000H ; ensures that your first instruction is placed where the CPU begins execution after reset.
- MOV A, #50 ; load 50 (decimal) value to accumulator
- MOV B,#20 ; load 25 (decimal) value to register B
- CLR C ; clear carry before SUBB
- SUBB A,B ; A = A – B – CY
- H1: SJMP H1 ; an infinite loop stops the microcontroller from running into garbage.
- END ; END stops the assembler from reading more code.

Write an ALP to perform multiplication of two 8-bit numbers

- ORG 0000H ; ensures that your first instruction is placed where the CPU begins execution after reset.
- MOV A, #12 ; load 12 (decimal) value to accumulator
- MOV B, #10 ; load 10 (decimal) value to register B
- MUL AB ; A = A x B A= Low byte of result B = High byte of result
- H1: SJMP H1 ; an infinite loop stops the microcontroller from running into garbage.
- END ; END stops the assembler from reading more code.

Write an ALP to perform multiplication of two 8-bit numbers

- ORG 0000H ; ensures that your first instruction is placed where the CPU begins execution after reset.
- MOV A, #50H ; load 50H to accumulator
- MOV B, #10 ; load 10H to register B
- DIV AB ; A = A / B A= Quotient B = Remainder
- H1: SJMP H1 ; an infinite loop stops the microcontroller from running into garbage.
- END ; END stops the assembler from reading more code.